

Research project - Who adopts Coding Agents on GitHub

Michał Sobczak

University of Warsaw

Warsaw, Poland

ms440009@students.mimuw.edu.pl

Abstract

With the emergence of AI agents such as Claude Code and GitHub Copilot, the software development process has evolved toward collaboration with autonomous assistants. This paper analyzes the relationship between a developer's level of experience and the way available agents are used. The study uses the **AIDev** dataset, released as part of the **MSR 2026 Mining Challenge**. The data cover the activity of over 72,000 users across more than 116,000 public repositories from December 2024 to August 2025, during which they collectively created 932,791 *pull requests*. To classify developers, both the data from the above dataset and information obtained via the **GitHub GraphQL API** were used. Users were divided into three groups: **Junior**, **Mid**, and **Senior**, within which an attempt was made to answer the following research questions:

- How does a developer's experience correlate with the frequency of using AI agents?
- Does human experience translate into a higher acceptance rate of agent-generated code?
- Do changes introduced by the junior group using AI require more corrections (measured by the number of comments) from the project maintainer?

1 Introduction

The evolution of AI tools in programming

In recent years, the development of artificial intelligence has significantly transformed the software development process. Initially, AI's role was limited to simple browser-based assistants, before evolving toward integration with IDEs in the form of code autocompletion systems. Today we are witnessing another breakthrough - agents such as **Claude Code** are gaining access to entire repositories, allowing them not only to fix bugs but also to independently implement new features.

Research objectives

The main goal of this paper is to investigate the correlation between a developer's level of experience and the way AI tools are used. The analysis attempts to verify whether developers at an early stage of their career path turn to AI agent support more often than experts, and how the acceptance rate of changes within *pull requests* varies depending on the user's level of advancement and the presence of an AI assistant. Additionally, the study examines whether experienced developers are able to describe a task to an agent precisely enough that the generated code requires fewer subsequent corrections.

Dataset

The research is based on the **AIDev** dataset, which is currently the largest and most up-to-date publicly available dataset of this type. A key advantage of this dataset is that it concerns commercial

projects, which significantly increases the reliability of the results in the context of the real job market.

During the initial analysis, I noticed that dividing users into the Junior, Mid, and Senior groups based solely on the dataset would not be reliable. Inferring a developer's level of experience solely from their interactions with AI agents could lead to erroneous results. Moreover, the number of *pull requests* created in this dataset exclusively by users is relatively small (6,618 rows compared to the overall total of 932,791 records), which would make a reliable comparison impossible. Therefore, in order to obtain a fuller picture of the studied individuals, I decided to acquire additional external information.

GitHub GraphQL API

In order to enrich the user profiles, I decided to obtain additional metrics directly from the GitHub platform. Manually checking so many profiles through a browser would be time-consuming, so I decided to use the GitHub API. Initially I tested the standard REST API, but it quickly turned out that it did not allow extracting sufficiently detailed information, and an additional obstacle was the restrictive hourly request limits. After several experiments, I abandoned this approach in favor of the GitHub GraphQL API. Thanks to lower limits and broader access to data, I was able to efficiently obtain the most important information about the studied group.

For each user, I obtained detailed data including the number of repositories they participated in, and the number of public and private *commits* over the last five years. Additionally, I included statistics on open *pull requests* and reported *issues*. Ultimately, I managed to gather data for 70,486 people out of the entire group of 72,189. Based on this data, I trained a random forest based on decision trees to predict the missing statistics. For this, I used the **Random Forest Regressor** model, which predicts continuous values for missing users. Numbers of *commits* or *reviews* performed are integers - the result is rounded to whole numbers.

It is worth noting that model validation (an 80/20 split) revealed limited predictive power of the AIDev dataset's features with respect to a user's overall GitHub activity (R^2 in the range of 0.05–0.15, and in some cases negative), suggesting that both data sources describe partially different aspects of activity, and the values imputed for missing users should be treated with some caution.

2 Methodology

For each user, I determined a set of parameters describing their activity in the GitHub repository

- The number of projects the user was involved in as a contributor.
- The number of *commits* made in public repositories over the last 5 years.

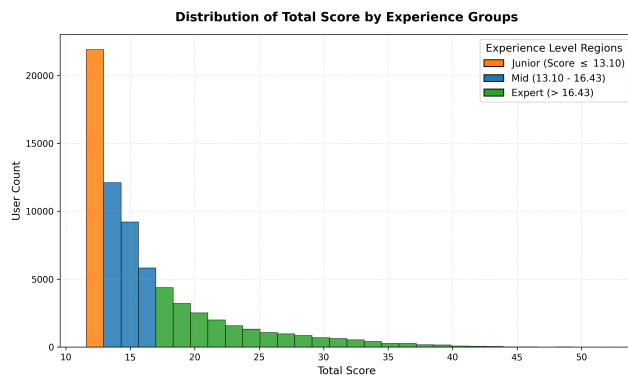
- The number of *commits* made in private repositories over the last 5 years.
- The number of *code reviews* performed over the last 5 years.
- The number of the user's own repositories created over the last 5 years.
- The number of *followers*.
- The maximum number of *stars* earned by a single, own repository.
- The maximum number of *forks* of a single, own repository.
- The account tenure on the GitHub platform (time since registration).
- The number of *pull requests* created, recorded in the **AIDev** dataset.
- The total number of commits recorded in the **AIDev** dataset.
- The number of code reviews performed, based on data from the **AIDev** dataset.
- The number of reported *issues* originating from the **AIDev** dataset.

2.1 Preliminary approach

The initial classification approach was based on an independent analysis of each parameter using a simple scoring system. For each of the studied features, the median and the 90th percentile were determined, which served as reference points in the scoring process. Then a function was applied assigning scores on a scale from 0 to 4 points, given by the following formula:

$$f(x) = \min(4, \max(0, y(P_{90}) + \frac{y(P_{90}) - y(P_{50})}{P_{90} - P_{50}}(x - P_{50})))$$

where P denotes the percentile function (i.e. P_{50} is the median), and $y(P_{50}) = 1$ and $y(P_{90}) = 3$. The final score for a user was a linear combination of the points obtained across all considered categories. Based on this calculated total value, I made the final division of users into three experience groups: Junior, Mid, and Senior.



Linear percentile-based scoring model

However, this method showed significant methodological flaws. First of all, it treated each parameter as completely independent, ignoring existing correlations. Moreover, this model was characterized by high arbitrariness - the percentile thresholds and the

final division into three groups were based on subjectively chosen values, which made reliable statistical verification impossible.

2.2 Target approach

In the final stage of the analysis, an unsupervised statistical approach was applied in the form of the *k-means* algorithm. The analyzed dataset consists exclusively of quantitative variables characterized by a significant spread of values (e.g. the number of public repositories ranges from 1 to 617,754). To mitigate this effect and ensure an equal influence of all variables on the clustering process, the data was subjected to a quantile transformation, bringing the distributions of the variables closer to normal, and standardization of features according to the formula:

$$z = \frac{x - \mu}{\sigma}$$

where μ denotes the arithmetic mean, and σ the standard deviation of a given feature.

In order to increase the stability and reliability of the results, outlier observations were removed from the dataset. For this, the **Isolation Forest** algorithm was used - an efficient algorithm for detecting anomalies, with the contamination parameter set at 1%. Then, in order to reduce noise and allow for a clear visualization of the multidimensional spaces, principal component analysis (**PCA**) was applied. Reducing the dimensionality to two principal components allowed for the preservation of key data structures while discarding 5% of the variance, treated as informational noise. The data prepared in this way served as input to the *k-means* algorithm.

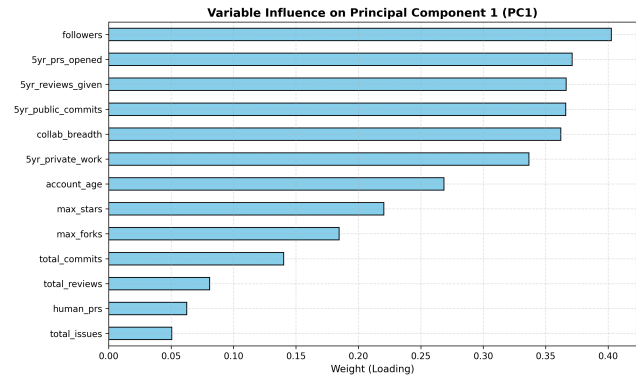


Figure 1: PCA PC1

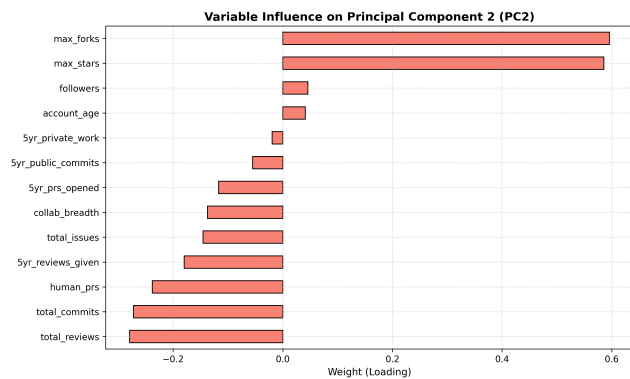
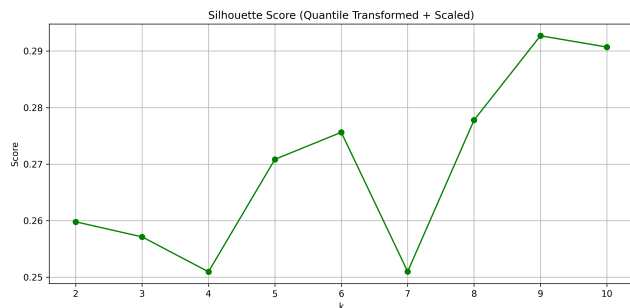


Figure 2: PCA PC2

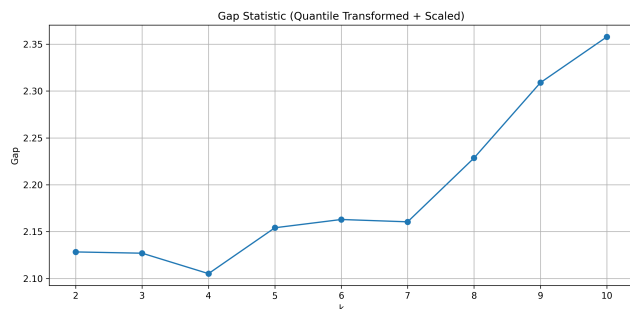
2.3 How to properly choose the parameter k?

To select the optimal number of clusters, I used two methods:

- **Gap Statistic:** A clustering quality assessment method that compares the total within-cluster variance of the real data with the expected variance for reference data generated from a uniform distribution.
- **Silhouette Method:** Measures how well each point fits into its cluster compared to neighboring clusters. This score oscillates between 1 and -1, where higher values indicate better separated and denser clusters.



Silhouette Method



Gap statistic

Ultimately, I chose the value $k = 3$. At $k = 4$, a noticeable decrease in the quality of the split is observed, and choosing a low value of the parameter prevents excessive fragmentation of the data and the creation of subgroups that are difficult to interpret. Additionally, in the business world this is the most common division, namely into: Juniors, Mids, and Seniors.

It is worth noting that the *k-means* algorithm performs clustering in an unsupervised manner - it determines clusters but does not assign them specific labels. In order to give the groups meaning and assign them appropriate names, principal component analysis was used. It allowed for identifying the attributes with the greatest influence on the differentiation of the data. On this basis, the profiles of the individual clusters were interpreted:

- **Juniors:** a group of users who achieved zero or minimal scores in the key variables.
- **Seniors:** a group comprising people with the highest values in the most important parameters.
- **Mid:** an intermediate group, characterized by moderate scores in the analyzed metrics.

3 Results

3.1 How does a developer's experience correlate with the frequency of using AI agents?

In order to assess the relationship between developers' level of experience and the frequency of using AI agents, a nonparametric Kruskal-Wallis test and post-hoc Mann-Whitney tests with Bonferroni correction were performed.

The Kruskal-Wallis test showed a statistically significant difference in the frequency of generating *pull requests* between the analyzed experience groups.

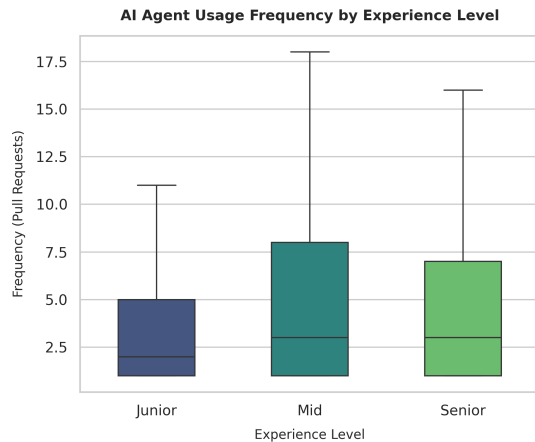
The post-hoc analysis showed the following relationships:

- The **Junior** group is characterized by a significantly lower frequency of using AI agents compared to the other groups.
- No statistically significant differences were found between the **Mid** and **Senior** groups.

Developers at the **Mid** and **Senior** level use AI agents the most, achieving the highest median usage.

Summary statistics

Level	N	Mean	SD	Mdn
Junior	53,327	9.25	47.97	2.0
Mid	16,192	21.94	407.51	3.0
Senior	1,948	17.76	97.93	3.0

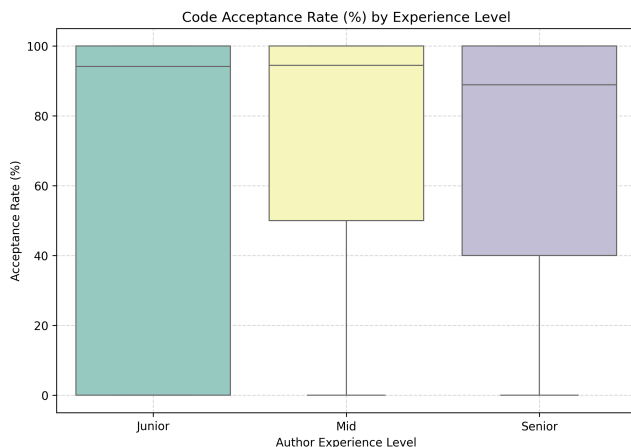


3.2 Does human experience translate into a higher acceptance rate of agent-generated code?

The analysis performed shows that a developer's level of experience affects the acceptance rate of code generated by AI agents. The Kruskal-Wallis test confirmed the existence of significant differences between the studied groups. A detailed analysis using post-hoc tests (Mann-Whitney U with Bonferroni correction) reveals the following relationships:

- Developers at the **Mid** level achieve the highest average code acceptance rate as well as the highest median.
- Code generated by **Seniors** shows a lower median acceptance rate among the other groups, but the tests did not show significant differences between the remaining groups ($p = 0.43$ for **Juniors** and $p = 1.00$ for **Mids**)

Based on the data, it can be concluded that human experience does not automatically translate into a higher code acceptance rate. Developers at the **Mid** level show the greatest effectiveness in accepting AI agent proposals.



Summary statistics

Level	N	Mean	SD	Mdn
Junior	51,554	63.33	44.05	94.12
Mid	12,857	71.55	38.31	94.44
Senior	1,945	67.44	40.03	88.89

3.3 Do changes introduced by the junior group using AI require more corrections (measured by the number of comments) from the project maintainer?

The analysis was based on counting comments left by maintainers within *pull requests*. In cases where the author of a *pull request* turned out to be a bot, the associated *issue* was checked, and its creator was treated as the actual author of the changes.

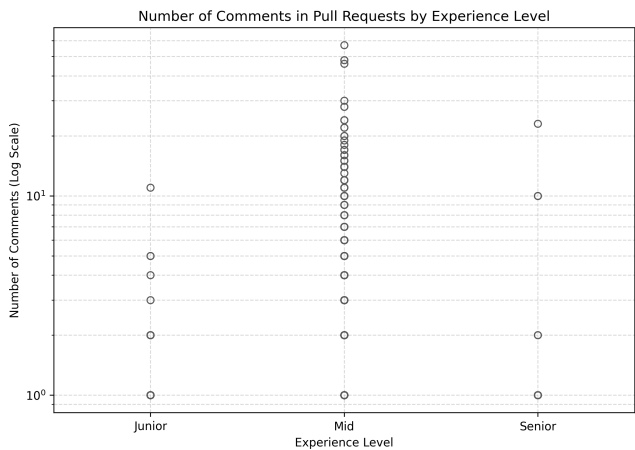
To verify this hypothesis, the **Kruskal-Wallis** test was used (robust to a large number of zero values in the data). The test confirmed the existence of statistical differences between all groups. Analysis using Mann-Whitney tests with Bonferroni correction showed that the differences are significant at every level of comparison:

- **Junior vs Mid:** $p < 0.001$
- **Junior vs Senior:** $p < 0.001$
- **Mid vs Senior:** $p < 0.001$

Code generated or supported by AI in the case of Juniors produced the lowest average number of comments. The highest rate was recorded in the **Mid** group. This result is somewhat surprising in the context of the hypothesis posed, and may result from the fact that Juniors create relatively simpler *pull requests*, while Mids undertake more complex changes that generate more remarks from maintainers - not necessarily due to lower code quality.

Summary statistics

Level	N	Mean	SD	Mdn
Junior	490,767	0.00014	0.02288	0.0
Mid	305,409	0.00701	0.29589	0.0
Senior	34,607	0.00170	0.14711	0.0



Acknowledgments

This work was created as part of the 2025/2026 research project under the supervision of prof. Krzysztof Stencel.